

第20章 异构计算集群编程：CUDA 流简介

20.1 背景

迄今为止，我们关注的是单主机单设备的异构计算系统编程。在高性能计算（HPC，High-Performance Computing）中，应用需要集群所有节点的汇总计算能力。如今，许多HPC集群中的每个节点都包含一个或多个主机（CPU）和一个或多个设备（GPU）。从历史上看，这些集群主要使用消息传递接口（Message Passing Interface, **MPI**）进行编程。MPI 是一组用于集群中进程间通信的 API 函数 ¹。

在本章中，我们将介绍 MPI/CUDA 联合编程，只介绍程序员需要理解的 MPI 概念，以便将异构应用扩展到集群环境的多个节点。我们特别关注域划分（domain partitioning）、点对点通信（point-to-point communication）和集体通信（collective communication）的概念，以及如何在多节点环境中扩展 CUDA 内核 ² ³。

虽然在 2009 年之前几乎没有排名靠前的超级计算机使用 GPU，但近年来对更高能效的需求使得 GPU 得到了快速采用。如今，世界上许多顶尖超级计算机的每个节点中都同时配置了 CPU 和 GPU。它们在 Green500 能效榜单中名列前茅，这验证了这种设计的有效性 ⁴。目前，用于计算集群的主导编程接口是 MPI（Gropp 等人，1999），它假设一种**分布式内存模型**，即进程通过发送消息来交换信息 ¹。当应用程序使用这些 MPI 通信函数时，程序员无需关心底层互连网络的细节。MPI 实现允许进程使用逻辑标识符（就像拨打电话号码一样）互相寻址：打电话时，我们只需拨号而不知道对方的具体位置或电路路由 ⁵。

在典型的 MPI 应用中，数据和计算任务被划分到各个进程中。如图20.1所示，每个节点可以包含一个或多个进程（图中以云图标识）。随着计算进行，进程间可能需要彼此的数据交换，这通过发送和接收 MPI 消息来满足 ³。在某些情况下，进程需要彼此同步并生成协作的集合结果，这就需要使用**集体通信API函数**（如 MPI_Barrier 等）来完成 ⁶。

20.2 运行示例

作为一个运行示例，我们将使用在第8章“Stencil”中介绍的三维（3D）模板计算（stencil computation）。假设该计算基于有限差分方法来求解描述热传导物理定律的偏微分方程，以模拟热传递。在迭代方法中（采用雅可比迭代 Jacobi 方式），每个格点在每个时间步的值由其前一时刻自身值与周围邻居值的加权和计算而来 ⁷ ⁸。为了提高数值稳定性，每个方向上还会使用多层间接邻居。这构成了一种高阶模板计算（higher-order stencil）。

在本示例中，我们假设每个方向取4个邻居点。如图20.2所示，一个格点的 x、y、z 坐标为 (i,j,k) 时，其 24 个邻居点可以列出如下（表示了四个负方向和四个正方向上的点） ⁹ ¹⁰。加上该点本身，共 25 个点（24 个邻居加自己）的数据用于计算下一时刻该点的值，因此这是一个**25点模板计算** ¹¹。

我们假设整个计算域被建模为一个结构化网格（structured grid），各坐标方向间距均匀，这使得可以使用一个三维数组来存储每个网格点的状态（如第8章所述）。物理距离可用网格间距变量来表示 ¹²。需要注意的是，这种数据结构与第18章“电势图”中使用的类似。图20.3示意一个矩形通风管道在三维数组中的表示，其中 x、y 方向为管道的横截面，z 方向为热流沿管道的方向 ¹²。

我们假设数据以行主序（row-major）方式存放在内存中，其中 x 维度最内层，y 次之，z 最高。也就是说，所有满足 $y=0$ 、 $z=0$ 的元素按其 x 坐标顺序连续存放。图20.4给出了一个小例子：该网格只有 16 个数据元素，x 维度2点，y 维度2点，z 维度4点。首先存放了所有 $y=0$ 、 $z=0$ 的 x 元素，然后是 $y=1$ 、 $z=0$ 的元素，接下来是 $y=0$ 、 $z=1$ 的元素等¹³¹⁴。

在使用计算集群时，通常会将输入数据划分为多个域（domain partitions），并将每个域分配给集群中的一个节点。在图20.3中，我们将三维数组沿 z 方向分为四个域分区：D0、D1、D2 和 D3，每个分区分配给一个 MPI 进程¹⁵。图20.4 展示了这一划分：第 0 层（ $z=0$ ）的四个元素被分配到第一个分区 D0，第 1 层（ $z=1$ ）分区到 D1，依次类推，这只是一个示意。在实际应用中，每个维度的点数通常为上百甚至上千。为了后续讨论，我们需要记住：所有处于同一 z 层的元素在内存中是连续存储的¹⁶。

20.3 MPI 基础

与 CUDA 类似，MPI 程序采用 SPMD（Single Program Multiple Data）并行模型，所有 MPI 进程执行相同的程序。MPI 系统提供了一系列 API 函数，用于建立进程间的通信系统。图20.5列出了初始化和关闭 MPI 通信系统所需的几个基本 MPI 函数。

图20.6 给出了一个简单的 MPI 程序的示例结构。在集群的登录节点上，用 `mpirun` 或 `mpiexec` 命令启动 MPI 应用时，需要指定该可执行文件¹⁷。在程序开始时，每个 MPI 进程首先调用 `MPI_Init()`（图20.6，第5行），初始化 MPI 运行时环境，这是针对所有进程的全局通信系统初始化¹⁸。初始化完成后，每个进程调用两个函数来准备通信：第一个是 `MPI_Comm_rank()`（第6行），它为每个进程返回一个唯一的整数标识，这就是 MPI 排序号（MPI rank）¹⁹²⁰。MPI 排序号类似于 CUDA 中线程的线性索引（如 `blockIdx.x*blockDim.x + threadIdx.x`），用于唯一标识进程在通信中的身份，相当于电话系统中的电话号码²¹。这些排名从 0 开始，到进程总数减 1 为止。

`MPI_Comm_rank()` 的第一个参数指定要查询的通讯域（communicator），最常用的是 `MPI_COMM_WORLD`，表示包括所有进程的默认通讯域²²。它返回的排序号被存储到第二个参数所指向的整型变量中（在示例中为变量 `pid`）。第二个函数是 `MPI_Comm_size()`（第7行），它返回通讯域中进程的总数²³。它的第一个参数同样是通讯域（通常是 `MPI_COMM_WORLD`），表示查询所有参与应用的进程数量，返回值存入其第二个参数指向的整型变量（示例中为 `np`）²⁴。

MPI 程序通常会检查实际创建的进程数是否满足需求。例如，示例中假定程序至少需要 3 个进程，因此在 `MPI_Comm_size()` 后检查 `np < 3`，如果不足，则由进程 0（即 `pid==0`）输出错误信息并调用 `MPI_Abort()` 终止通信系统²⁵²⁶。这种设计类似于 CUDA 中一个线程负责打印错误信息的做法²⁷。值得注意的是，`MPI_Abort()` 与其他 MPI 函数一样，第一个参数也是通讯域，第二个参数是错误代码²⁸。

下一步，根据进程 `pid` 来决定其角色：示例中用 `np-1`（最后一个）进程作为数据服务器，其余进程作为计算进程。如果 `pid < np-1`，则调用 `compute_process()` 进行计算，否则最后一个进程调用 `data_server()` 提供 I/O 服务²⁹³⁰。这与 CUDA 中根据线程 ID 分配不同工作任务的模式类似³¹。

应用计算完成后，每个进程调用 `MPI_Finalize()`（第16行），释放所有 MPI 通信资源，然后返回主程序结束³²。

20.4 MPI 点对点通信

MPI 支持两种主要的通信方式：点对点通信和群体通信。点对点通信指一个源进程和一个目标进程之间的通信。源进程调用 `MPI_Send()` 函数发送消息，目标进程调用 `MPI_Recv()` 接收消息，就像一个电话呼叫和应答一样³³。

图20.7 展示了 `MPI_Send()` 的使用语法：第一个参数是指向要发送数据起始位置的指针；第二个参数是发送数据元素的个数；第三个参数指定每个元素的数据类型（`MPI_Datatype`，如 `MPI_FLOAT` 表示单精度浮点）³⁴；第四个参数是目标进程的 MPI 排序号；第五个参数是用户定义的标签（tag），用于对同一源进程发送的消息进行分类；第六个参数是通讯域（通常是 `MPI_COMM_WORLD`），用于指定排序号的上下文³⁶。

图20.8 展示了 `MPI_Recv()` 的语法：第一个参数是指向接收缓冲区的指针；第二个参数是允许接收的最大元素个数；第三个参数是接收数据类型；第四个参数是发送源进程的排序号；第五个参数是期望的标签值。如果接收方不关心标签，可以使用 `MPI_ANY_TAG`，表示接受任何标签的消息³⁷³⁸。第六个参数是通讯域，第七个参数是状态（`MPI_Status`）结构的指针，用于返回通信状态信息。

下面我们以数据服务器为例说明点对点通信。在真实应用中，数据服务器进程通常负责为计算进程执行数据输入/输出。然而由于 I/O 操作过于复杂（与系统相关），我们在此示例中简化处理：数据服务器不从文件读取，而是通过随机数初始化数据并分发给计算进程。图20.9 展示了数据服务器函数 `data_server()` 的前半部分³⁹。该函数有四个参数：前三个分别是三维网格在 `x`、`y`、`z` 方向上的大小（`dimx`、`dimy`、`dimz`），第四个参数是迭代次数 `nreps`⁴⁰。

在图20.9中，第2行声明变量 `np` 将保存运行应用的总进程数；第3行调用 `MPI_Comm_size()` 获得进程总数；第4行初始化一些辅助变量：`num_comp_nodes = np - 1` 表示计算进程的数量（保留一个进程作为数据服务器），`first_node = 0` 表示第一个计算进程的排序号，`last_node = np - 2` 表示最后一个计算进程的排序号⁴¹。随后，第5行计算整个网格中点的总数 `num_points = dimx * dimy * dimz`，第6行计算存储这些点所需的字节数 `num_bytes = num_points * sizeof(float)`⁴²。第7行声明两个浮点指针 `input` 和 `output`，用于分别指向输入数据缓冲区和输出数据缓冲区⁴³。

第8行和第9行分别用 `malloc()` 为 `input` 和 `output` 缓冲区分配内存（长度为 `num_bytes`）。如果分配失败，程序在第11行输出错误信息并在第12行调用 `MPI_Abort()` 终止⁴⁴。第14行调用 `random_data()`（假定存在的函数）对输入缓冲区进行随机初始化⁴⁵。

接下来，第15行计算每个进程需要处理的“边界”点个数：`edge_num_points = dimx * dimy * ((dimz / num_comp_nodes) + 4)`，第16行计算“内部”点个数：`int_num_points = dimx * dimy * ((dimz / num_comp_nodes) + 8)`⁴⁶。这两个值考虑了每个方向上 4 个额外的邻居层（halo cells），其中边界进程只有一侧有邻居（故加4），内部进程两侧都有邻居（故加8）。第17行将 `send_address` 指向输入网格数据的起始位置⁴⁷。

第18行开始向各个计算进程发送数据：首先，把第一个分区的数据发送给排序号为 `first_node`（即0）的进程⁴⁸。由于第0进程只需要右侧邻居的数据，所以此时发送的数据长度为 `edge_num_points`（相当于 `(dimz / num_comp_nodes) + 4`）⁴⁹。第19行将 `send_address` 指针向后移动 `dimx * dimy * ((dimz / num_comp_nodes) - 4)`，为下一个发送做准备⁵⁰。第20行在一个循环中对内部进程（从 1 到 `last_node-1`）依次发送数据，使用长度 `int_num_points`（即 `(dimz / num_comp_nodes) + 8`）⁵¹⁵²。第22行更新 `send_address`，每次增加一个完整分区的点数 `dimx * dimy * (dimz /`

`num_comp_nodes`)⁵³。最后,第23行向最后一个进程 `last_node` 发送边界分区的数据,长度为 `edge_num_points`⁵¹⁵⁴。图20.9 第24行标记发送操作结束。到此,数据服务器部分的工作完成(图中显示的“数据服务器代码(第1部分)”)⁵⁵⁵⁶。

20.5 计算与通信重叠

一种简单的计算策略是:每个计算进程对其整个分区进行计算,然后交换边界(halo)数据,再继续下一步。这个策略的问题在于系统不能同时利用计算和通信:在一个阶段所有进程都在计算时网络空闲,下一阶段都在通信时计算资源空闲⁵⁷。理想情况下,我们希望在计算过程中交错进行通信,以同时利用 CPU/GPU 和网络资源。为此,可以将每次迭代的计算过程拆成两个阶段⁵⁸:

- **阶段1**:各进程先计算其分区边界上的数据——这些数据将成为下一代中其邻居所需的 halo 单元。例如,假设每侧需要 4 层 halo(如前文所设)。图20.12 用色块示出了这一过程:进程 *i* 在第一阶段计算自己的左 4 层和右 4 层边界数据(进程0只计算右侧边界,最后一个进程只计算左侧边界)⁵⁹。这样做的目的是让这些边界切片先行计算完成,然后可以与邻居通信,而计算进程同时去计算分区内部的其他点,从而让计算与通信并行⁶⁰⁶¹。
- **阶段2**:每个计算进程同时进行两项操作:一是将新计算出的边界数据复制到主机内存,并用 MPI 发送给左右邻居;二是继续计算分区内部其余的点。如果通信所需时间低于内部计算时间,那么通信延迟就可以被隐藏,从而实现计算与通信的重叠⁶²⁶³。

为支持阶段2的并行操作,需要使用 CUDA 的两个高级特性:固定(页锁定)内存和 CUDA 流(streams)。固定内存是指用 `cudaHostAlloc()` 分配的不可换出(pinned)内存⁶⁴。在图20.11中,第21-24行用 `cudaHostAlloc()` 分配了左右边界和左右 halo 的主机缓冲区。这些缓冲区作为“跳板缓冲区”(bounce buffers):左/右边界缓冲区用于从设备复制相应边界数据,然后作为 `MPI_Send()` 的源;左/右 halo 缓冲区作为 `MPI_Recv()` 的接收缓冲区,然后再将接收到的数据复制回设备内存⁶⁴。使用 `cudaHostAlloc()` 分配固定内存可以避免内存被操作系统换出,确保 DMA 直接内存传输(DMA)时数据安全⁶⁵⁶⁶。

第二个高级特性是 CUDA 流(CUDA Streams)。流是一系列按序执行的 CUDA 操作序列。通过给 `cudaMemcpyAsync()` 或 kernel launch 指定流参数,可以将复制或计算操作投递到不同的流中执行⁶⁷⁶⁸。同一流内的操作会按照顺序依次执行,但不同流的操作可以并行进行,无需相互等待⁶⁸。图20.11 第25-27行创建了两个 CUDA 流 `stream0` 和 `stream1`⁶⁹。这样,我们就可以让用于边界切片的内存复制和发送放在一个流中(例如 `stream0`),而内部区域的计算内核放在另一个流中(例如 `stream1`),实现真正的并行。

图20.13 和后续代码展示了计算进程的完整实现:

- 图20.13 (第3部分)中,前几行初始化 MPI 排序号、邻居排序号以及一些偏移量(offset)变量,用于确定需要计算或交换的网格区域⁷⁰⁷¹;然后调用 `upload_coefficients()` 将常量(stencil系数)上传到 GPU 常量内存(省略细节)⁷²。接着设置多个偏移量 `left_stage1_offset`、`right_stage1_offset`、`left_halo_offset`、`right_halo_offset` 等,用于分离阶段1和阶段2需要处理的内存区域⁷³。例如,`left_stage1_offset` 为 0,指向分区最左边的4层 halo+4层边界+4层内部点(共 12 层);`right_stage1_offset` 指向右侧 12 层的起始位置⁷⁴⁷⁵。然后调用 `MPI_Barrier(MPI_COMM_WORLD)` 同步所有进程,确保所有进程都已接收到输入数据并准备好进行迭代⁷⁶。

之后进入迭代循环:第39-40行分别通过调用 CUDA kernel (`call_stencil_kernel`) 来计算左侧和右侧边界所需的 4 层数据,这两个内核都在 `stream0` 中依次执行⁷⁷。第41行在 `stream1` 中调用内核计算其余的内部

区域,覆盖剩余 `dimz-8` 层的数据⁷⁷。由于这三个内核被投递到不同的流,它们可以并行执行,其中 `stream1` 内的内部计算内核与 `stream0` 内的边界计算内核同时进行⁷⁷。

图20.15 (第4部分)展示了计算过程中的通信部分。第42-43行使用 `cudaMemcpyAsync` 将左右边界数据从设备复制到主机缓冲区 `h_left_boundary` 和 `h_right_boundary` (两个操作都在 `stream0` 上执行,并等待先前在同一流中的两个内核完成)⁷⁸。第44行调用 `cudaStreamSynchronize(stream0)` 强制等待 `stream0` 中的所有操作结束,确保边界数据已经到达主机内存⁷⁹。然后第45-46行调用 `MPI_Sendrecv()` 进行数据交换:进程将左边界切片发送给左邻居,同时接收右邻居发来的右侧 halo 切片;第46行则相反,发送右边界并接收左侧 halo^{80 81}。在进程0中,由于没有左邻居,左邻居的排名被设置为 `MPI_PROC_NULL`⁸²。在进程最后一个 `np-2` 中,由于没有右邻居,右邻居设置为 `MPI_PROC_NULL`。因此 MPI 自动处理边界情况,程序无需在此做分支判断⁸³。

第47-48行将接收到的左侧和右侧 halo 数据分别异步复制回 GPU (在 `stream0` 中)⁸⁴。此时, `stream1` 内执行内部计算的内核(第41行)也在并行运行。接着第49行调用 `cudaDeviceSynchronize()`,等待所有设备操作完成⁸⁵。此时, `d_output` 缓冲区中已包含完整的当前迭代结果:左侧 halo (来自左邻居)、左边界、内部数据、右边界、右侧 halo (来自右邻居)⁸⁶。第50-51行交换 `d_input` 和 `d_output` 指针,为下一迭代准备。循环继续,直至完成设定的迭代次数⁸⁷。最后(图20.17,第5部分),第53行再次调用 `MPI_Barrier` 确保所有进程完成迭代;第54-56行交换一次 `d_input/d_output` (撤销上次交换使输出正确);第57行将最终输出从设备复制到主机;第58行将输出发送回数据服务器进程;第59行再次同步;最后释放内存并返回^{88 89}。

20.6 MPI 群体通信

本章前面已经展示了一个 MPI 群体通信示例: `MPI_Barrier()`。其他常用的群体通信函数还包括广播 (broadcast)、归约 (reduce)、汇集 (gather) 和分发 (scatter) 等⁹⁰。例如, `MPI_Barrier()` 用于在协作任务中保证所有进程都准备就绪后再继续下一步。我们这里不详细介绍其他函数,但值得指出, MPI 的群体通信函数通常经过高度优化,使用它们往往比用多次点对点发送/接收实现相同功能更高效、更简洁⁹¹。

在我们的示例中,数据服务器在程序末尾使用了两次 `MPI_Barrier()` (图20.18,第2部分)。其中 `MPI_Barrier(MPI_COMM_WORLD)` (第24行)等待所有计算进程完成计算并将结果发送过来⁹²;随后第26-27行循环接收所有进程发送回来的输出分块⁹³;第28行调用 `store_output()` 将结果存入外部存储⁹⁴。最后在第29-30行释放内存⁹⁵。

20.7 支持 CUDA 的 MPI

现代 MPI 实现已经支持 CUDA 模型,能够尽量减少 GPU 之间通信的延迟。例如, MVAPICH2、IBM Platform MPI 和 OpenMPI 等 MPI 库均支持直接在节点间发送 GPU 内存之间的消息⁹⁶。这就消除了发送 MPI 消息前需要将数据从设备复制到主机、以及接收消息后再将数据复制回设备的步骤,简化了主机代码和数据布局⁹⁷。

以我们的例子为例,使用 CUDA-感知 MPI (CUDA-aware MPI) 后,不再需要对 halos 数据进行主机固定内存分配和异步传输。也就是说,图20.11 中第21-24行的固定内存分配可以去除⁹⁸。此外,在 MPI 接收后也不再需要异步地将 halo 数据从主机复制回设备,因此图20.15 第42-43行的数据传输可以省略⁹⁹。同时,由于 CUDA-感知 MPI 允许直接使用设备地址作为消息缓冲区,我们将 `MPI_Sendrecv()` 调用改为直接读写 GPU 内存。具体地,用设备指针替代之前的主机缓冲区地址¹⁰⁰。例如,图20.19 所示的语句直接使用 `d_output` (设备地址)作为发送和接收缓冲区¹⁰¹。这样就完全移除了通信过程中的额外内存拷贝步骤。

20.8 总结

本章介绍了在具有异构节点（每节点包括 CPU 和 GPU）的高性能计算集群上，如何采用 MPI/CUDA 联合编程的基本模式。所有 MPI 进程运行相同的程序，但不同进程可以执行不同的控制路径和函数，来完成各自的角色（如示例中的数据服务器与计算进程）。我们使用模板模式（stencil）展示了计算进程如何交换边界数据，并演示了如何利用 CUDA 流和异步数据传输来实现计算与通信的重叠。我们还说明了如何使用 `MPI_Barrier()` 来确保所有进程在数据交换前同步就绪。最后简要介绍了 CUDA-感知 MPI，使得在不同节点的 GPU 之间直接交换数据更简便。总体而言，MPI 模型（SPMD、MPI 排序号和屏障同步等）与 CUDA 模型有很多相似之处。掌握了一个模型后，可以快速迁移到另一个模型。希望读者能够以本章为基础，进一步学习更高级的 MPI 功能和模式。